Lab 3: Wi-Fi Coverage Algorithms — Greedy vs. Brute Force

Course: COMP 368 — Applied Algorithms **Dataset:** Kenyon College campus buildings (OpenStreetMap via Overpass Turbo)

---

1. Problem Overview

The goal of this lab is to figure out the minimum number of Wi-Fi access points (APs) needed to cover every building on a college campus. Each building is represented as a point in a flat, meter-scale coordinate system (x_m, y_m), and each AP covers any building within a radius R meters of where it's placed.

This is actually a well-known problem in computer science called **Set Cover**. In our version: - Each **building** is an element that needs to be covered - Each **possible AP placement** defines a set of buildings it can reach - The goal is to select the fewest sets (AP locations) whose union covers all elements (buildings)

Set Cover is NP-hard, meaning no known algorithm can solve it optimally in polynomial time for large inputs. The brute-force approach that checks every possible subset of placements works fine when there are only a handful of buildings, but it becomes completely impractical as the input grows. The whole point of this lab is to see how well greedy heuristics hold up in comparison — and to actually measure that on real campus data.

The dataset used throughout this lab is `kenyonout.csv`, which contains **88 Kenyon College buildings** converted to a local coordinate system using OpenStreetMap data. A 15-point subset (`small_test.csv`) was used wherever brute-force comparisons were needed.

---

2. The Brute-Force Baseline

How it works

The brute-force solver treats every building location as a potential AP placement. It then iterates over every possible non-empty subset of those locations using a bitmask (from `1` to `2^n -`

`1`), checking whether each subset fully covers all demand points. The smallest subset that achieves full coverage is returned as the answer.

```
for each subset S of building locations:
    if S covers all n buildings:
        if |S| < best_size:
            best = S
```

One important implementation detail is the **timeout guard**: on every iteration, the solver checks how much time has elapsed. If the user-specified timeout (via `--timeout-ms`) is reached, the solver immediately returns the best solution found so far instead of continuing. This makes it safe to call on moderately-sized inputs without waiting indefinitely.

Why it's correct but slow

The brute-force approach is provably correct because it considers every possible combination of AP placements. It is guaranteed to find the optimal solution. The problem is that the number of subsets grows as **2^n**, which is exponential. Even with pruning (skipping subsets larger than the current best), the growth is overwhelming:

n (buildings)	Subsets to check	Brute-Force Runtime
5	31	< 0.1 ms
10	1,023	< 0.1 ms
15	32,767	1.0 ms
18	262,143	7.7 ms
20	1,048,575	30.8 ms
22	4,194,303	130.5 ms
24	16,777,215	507.5 ms
26	67,108,863	2,031 ms
28	268,435,455	8,005 ms (timeout triggered)

(All tests at radius = 300 m)

The jump from n=26 to n=28 is striking — it goes from 2 seconds to 8 seconds, and at n=28 the 10-second timeout is already being hit. By contrast, the full 88-building campus dataset would require checking over **3 × 10^26** subsets — a number so large it would take longer than the age of the universe. This is precisely why greedy algorithms exist.

---

3. Greedy Algorithms

All three greedy algorithms follow the same general loop:

1. Start with zero APs and all buildings uncovered
2. Each round, make one local decision to place an AP somewhere
3. Mark newly-covered buildings and repeat until full coverage (or the `--max-aps` cap is hit)

The difference between them is **what "a good local decision" means**.

3.1 Greedy 1 — Maximum Coverage (Classic Set Cover Heuristic)

Description

At every step, scan all building locations as candidate AP positions. For each candidate, count how many currently-uncovered buildings it would reach. Place the AP at whichever candidate covers the most uncovered buildings. Break ties by taking the first maximum found.

```
while uncovered buildings remain:
    best_candidate = argmax over all buildings j:
                         count of uncovered buildings within radius of j
    place AP at best_candidate
    mark newly-covered buildings
```

Why I expected it to work

This is the textbook greedy approximation for Set Cover. The intuition is simple: if you always pick the single choice that eliminates the most remaining work, you tend to reach a solution quickly without much wasted effort. Skiena specifically discusses this heuristic and shows it achieves an O(log n) approximation ratio — meaning it uses at most O(log n) times the optimal number of APs. In practice, it often matches or comes very close to the optimum on well-structured data like campus buildings, which tend to cluster around academic and residential areas.

The main weakness is that it makes decisions based purely on the current round. It can't "see ahead" — so if two buildings are isolated and far from everything else, it might choose a central AP that covers many nearby buildings first, only to realize later it still needs two separate APs for the outliers anyway.

3.2 Greedy 2 — Farthest Uncovered Point First

Description

Each round, find the uncovered building that is farthest from any existing AP. Place the next AP at that building's location. On the very first round (when there are no APs yet), the algorithm just picks the first uncovered building.

```
while uncovered buildings remain:
    farthest = argmax over uncovered buildings i:
                  min distance from i to any existing AP
    place AP at farthest
    mark newly-covered buildings
```

Why I expected it to work

The key insight here is to deal with isolated outlier buildings early. If there's a building at the edge of campus that is 600 meters from everything else, a coverage-maximizing algorithm might leave it for last and end up needing an extra dedicated AP for it anyway. By tackling the "hardest" point first, this algorithm ensures that isolated buildings drive AP placement, and nearby buildings get covered as a bonus.

I expected this to potentially use more APs than Greedy 1 on dense inputs, because placing an AP at a faraway isolated building is wasteful if that building's neighbors are sparse. But I thought it might perform better on campuses with a spread-out layout where a few outlier buildings exist.

Observed behavior

The results show that Greedy 2 consistently used more APs than Greedy 1 across all radii tested. The farthest-first strategy forces APs into sparse, remote areas early on, which means the first few APs don't cover many buildings. Later rounds have to fill in the dense center, undoing most of the advantage. It still achieves full coverage, just less efficiently.

3.3 Greedy 3 — Centroid of Uncovered Points

Description

Each round, compute the centroid (average x and average y) of all currently-uncovered buildings. Find the uncovered building that is closest to that centroid, and place the AP there. Repeat.

```
while uncovered buildings remain:
    cx, cy = average position of all uncovered buildings
    snap = argmin over uncovered buildings i: distance(i, centroid)
    place AP at snap
    mark newly-covered buildings
```

The "snap to nearest demand point" step is important. The raw centroid often falls in empty space (between buildings, on a road, or even off-campus), so placing the AP exactly at the centroid would cover very few buildings. Snapping to the nearest uncovered building ensures the AP always sits on a real location.

Why I expected it to work

The centroid minimizes the total squared distance from a point to all other points. So placing an AP at (or near) the centroid of uncovered buildings is a reasonable way to reduce the "average distance to the nearest AP" metric. My expectation was that this strategy would produce compact, roughly-optimal clusters and use a similar number of APs to Greedy 1.

Observed behavior

In practice, Greedy 3 performs comparably to Greedy 2 — better than nothing, but worse than Greedy 1 across most radii. The centroid strategy tends to place APs in the middle of dense clusters, which works well initially but leaves outlier buildings for later. Each time an AP is placed in the center of the remaining uncovered group, it chips away at the core, but the stragglers at the edges remain uncovered and each require their own AP. Greedy 1's coverage-count approach naturally handles this, because a central AP covering 10 buildings beats a centroid AP covering 5.

---

4. Experimental Results

4.1 Small Instance — Brute Force Feasibility Test

The following table uses the 15-building subset at varying radii. Brute force gives the optimal answer; the greedy algorithms are compared against it.

Radius (m)	Brute Force (APs)	Greedy 1 (APs)	Greedy 2 (APs)	Greedy 3 (APs)	G1 Optimal?	G2 Optimal?	G3 Optimal?
150	6	6	6	7	Yes	Yes	No (+1)

Radius (m)	Brute Force (APs)	Greedy 1 (APs)	Greedy 2 (APs)	Greedy 3 (APs)	G1 Optimal?	G2 Optimal?	G3 Optimal?
200	5	6	6	6	No (+1)	No (+1)	No (+1)
300	3	3	4	3	Yes	No (+1)	Yes
400	3	3	3	3	Yes	Yes	Yes

All algorithms achieve 100% coverage at every radius. What the table shows is that **none of the greedy algorithms are always optimal** — each one produces a suboptimal result at at least one radius setting. Greedy 1 generally performs the best, hitting optimal in 3 out of 4 cases. Greedy 2 and Greedy 3 each miss optimal twice.

The runtime difference is dramatic:

Algorithm	Avg Runtime (15 pts)
Brute Force	~1.2 ms
Greedy 1	< 0.1 ms
Greedy 2	< 0.1 ms
Greedy 3	< 0.1 ms

For 15 points, brute force is still fast enough to be usable (about 12× slower than greedy). But as the scalability table in Section 2 shows, that advantage evaporates completely by the time n reaches 28.

4.2 Large Instance — Full Campus (88 Buildings)

Brute force is not feasible here (would require examining ~3 × 10^26 subsets). All three greedy algorithms run and produce full coverage.

Radius (m)	Greedy 1 (APs)	Greedy 2 (APs)	Greedy 3 (APs)	G1 Runtime	G2 Runtime	G3 Runtime
50	49	54	54	0.4 ms	0.1 ms	0.0 ms
75	31	37	37	0.3 ms	0.1 ms	0.0 ms
100	20	24	27	0.1 ms	0.0 ms	0.0 ms

Radius (m)	Greedy 1 (APs)	Greedy 2 (APs)	Greedy 3 (APs)	G1 Runtime	G2 Runtime	G3 Runtime
150	12	17	17	0.1 ms	0.0 ms	0.0 ms
200	11	13	12	0.1 ms	0.0 ms	0.0 ms
300	5	8	8	0.0 ms	0.0 ms	0.0 ms

All three algorithms achieve 100% coverage at every radius tested. Greedy 1 consistently requires the fewest APs — as much as 38% fewer than Greedy 2 or 3 at larger radii. Greedy 2 and Greedy 3 are nearly identical to each other across the board.

Effect of radius on AP count: As expected, larger radii require fewer APs. Going from R=50 to R=300 reduces Greedy 1's AP count from 49 down to 5 — roughly a 10× reduction. This makes sense geometrically: doubling the radius roughly quadruples the area covered per AP.

4.3 AP Count vs. Radius Summary (Greedy 1)

Radius (m)	APs Required (G1)	Avg buildings/AP
50	49	1.8
75	31	2.8
100	20	4.4
150	12	7.3
200	11	8.0
300	5	17.6

The "average buildings per AP" metric shows how efficiently APs are being utilized. At R=50m, most APs are essentially dedicated to a single isolated building. By R=300m, each AP is covering over 17 buildings on average, meaning the campus buildings are mostly within 300m of each other.

---

5. Why Brute Force Fails at Scale

The core reason is simple: brute force has **exponential time complexity**. For n buildings, there are 2^n subsets to evaluate. Even with pruning (skipping subsets larger than the current best), the worst-case and typical-case behavior grows exponentially.

The experimental evidence is clear:

n	Brute Force Runtime	Greedy 1 Runtime
15	1.0 ms	< 0.1 ms
18	7.7 ms	< 0.1 ms
20	30.8 ms	< 0.1 ms
22	130.5 ms	< 0.1 ms
24	507.5 ms	< 0.1 ms
26	2,031 ms	< 0.1 ms
28	8,005 ms (timeout hit)	< 0.1 ms
88	infeasible (years)	0.4 ms

Between n=20 and n=24, the runtime roughly increases 16× (consistent with 2^4 = 16 for four additional points). This is the hallmark of exponential growth. Meanwhile, all three greedy algorithms run in under 0.4 ms regardless of input size, because they operate in $O(n^2)$ time — scanning all n candidates once per AP placed, for at most n rounds.

To put the 88-building case in perspective: the brute force algorithm would need to evaluate 2^88 ≈ 3.09 × 10^26 subsets. Even at a billion evaluations per second, that would take roughly 9.7 × 10^9 years — about twice the current age of the universe.

---

6. Analysis and Discussion

Which greedy algorithm is best?

Based on the experiments, **Greedy 1 (Maximum Coverage)** is the clear winner. It uses the fewest APs at every radius tested on the full campus, and it matches or closely approximates the optimal solution on the small test set. The O(log n) approximation bound that Skiena describes

is visible in practice: Greedy 1 is consistently close to optimal even though it makes no attempt to look ahead.

Greedy 2 and Greedy 3 are comparable to each other in performance but meaningfully worse than Greedy 1. This makes sense in retrospect: both of them prioritize geometry (where points are located spatially) rather than coverage count (how many buildings would be covered). On a campus where buildings cluster around quads, dining halls, and dorms, coverage-based reasoning is more effective.

Where the greedy algorithms fail

The small instance test at R=200m reveals an important failure case for Greedy 1: it placed 6 APs while the optimal was 5. This happened because the algorithm made a locally good choice in round 1 that foreclosed a more globally efficient arrangement. A human looking at a map might see that one carefully placed AP could cover a cluster the greedy algorithm split across two rounds.

Greedy 3 has a specific structural weakness: it works poorly when the uncovered buildings form two well-separated clusters. The centroid of two clusters falls between them, and the snapped AP serves only the closer cluster. A smarter strategy would detect the clusters and handle them independently — something Greedy 1 naturally does without any special-casing.

Limitations of the model

The simplified disk-coverage model used here ignores several real-world factors:

- **Building materials and walls:** Concrete and metal significantly attenuate Wi-Fi signals. An AP theoretically covering 100m in open air might only reach 20-30m through walls.
- **Vertical coverage:** The model is 2D, but buildings have multiple floors. A single AP rarely covers an entire multi-story building.
- **Channel interference:** Multiple overlapping APs on the same channel reduce each other's effective throughput, even if geometric coverage looks fine.
- **AP placement constraints:** In practice, APs can only be mounted in certain locations (ceilings, walls, poles). The model assumes any building centroid is a valid placement.

Despite these limitations, the model is valuable for isolating the algorithmic question: given a placement problem, how well do greedy strategies approximate the optimal? The answer is: quite well for Greedy 1, reasonably well for the others, and far better than brute force at any realistic scale.

---

7. Conclusion

This lab demonstrated both the power and the limits of greedy algorithms for the Wi-Fi Set Cover problem. The brute-force solver is correct by construction but collapses entirely at even modest input sizes — by n=28 it was already timing out at 8 seconds, and applying it to 88 buildings is not even theoretically feasible in any practical sense.

The three greedy algorithms, by contrast, solve the full 88-building campus instance in under half a millisecond with 100% coverage. Greedy 1 (Maximum Coverage) stands out as the most effective heuristic: it consistently uses the fewest APs and closely tracks the brute-force optimal on small inputs. Greedy 2 (Farthest-First) and Greedy 3 (Centroid-Snap) are both functional and fast, but they use 20-40% more APs than Greedy 1 across most settings.

The key takeaway is that smart local decisions — specifically, always picking the AP placement that covers the most remaining buildings — compound into a globally efficient solution. This is exactly what Skiena means when he says greedy algorithms are "surprisingly effective in practice." They don't guarantee optimality, but for real-world inputs like campus building maps, they get very close, very quickly.